STM32H750 电赛智能车 – 原理与实现

3.2:视觉模块通讯配置:
官方手册查看与通信参数设置

InnoLegend 技术团队

目 录

CONTENTS

1 串口通信概述

2 OpenMV串口通信的实现

3 K210串口通信的实现

第一部分

串口通信概述

Introduction to development environment

1.1 串口通信

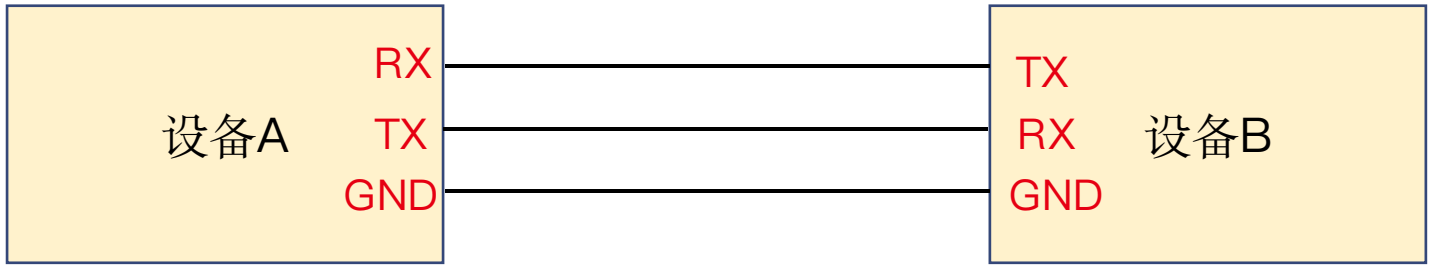
InnoLegend

串口通信是一种按位（bit）顺序传输的通信方式，最常见的异步串口通信协议是 UART (Universal Asynchronous Receiver/Transmitter)，即“通用异步收发器”

UART协议的核心参数：

- 波特率（Baud Rate）：
  - 规定了数据发送和接收的速度，通信双方的波特率必须完全一致
  - 常见值：9600、115200
- 数据帧 (Data Frame)：
  - 串口通信的最小单元
  - 通常包含帧头、帧尾、数据位、校验位

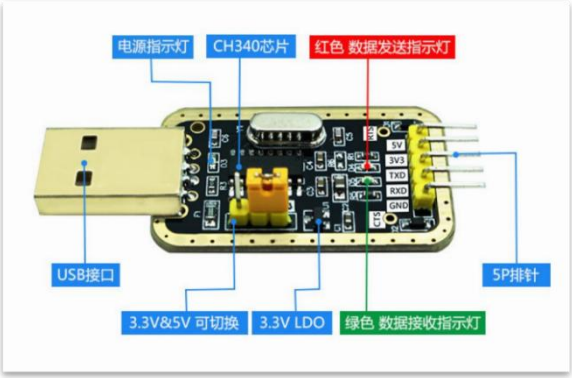
串口通信的接线方式：



1.2 串口通信调试工具

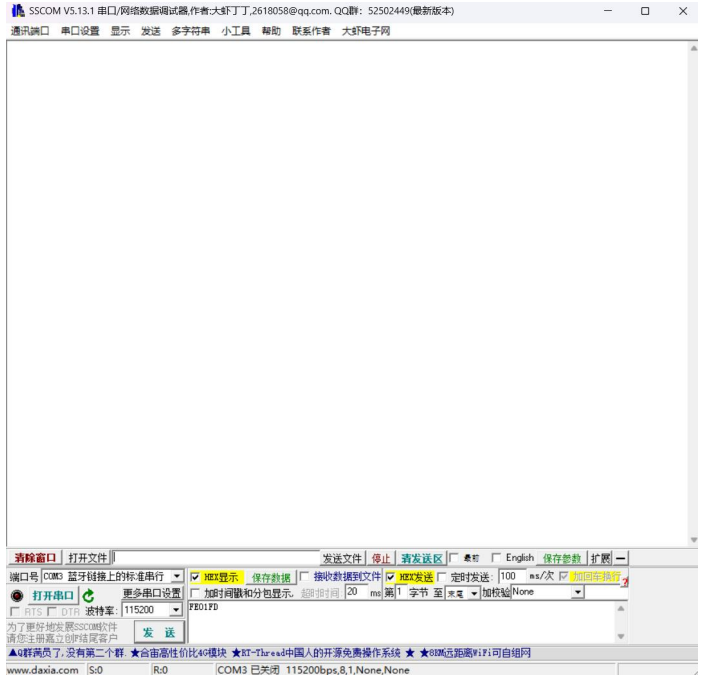
InnoLegend

USB-TTL模块



让电脑能够通过USB口与单片机等电子设备的TTL电平串口进行通信

串口调试助手



下载地址：
https://soft.3dmgame.com/down/247967.html

第二部分

OpenMV串口通信的实现

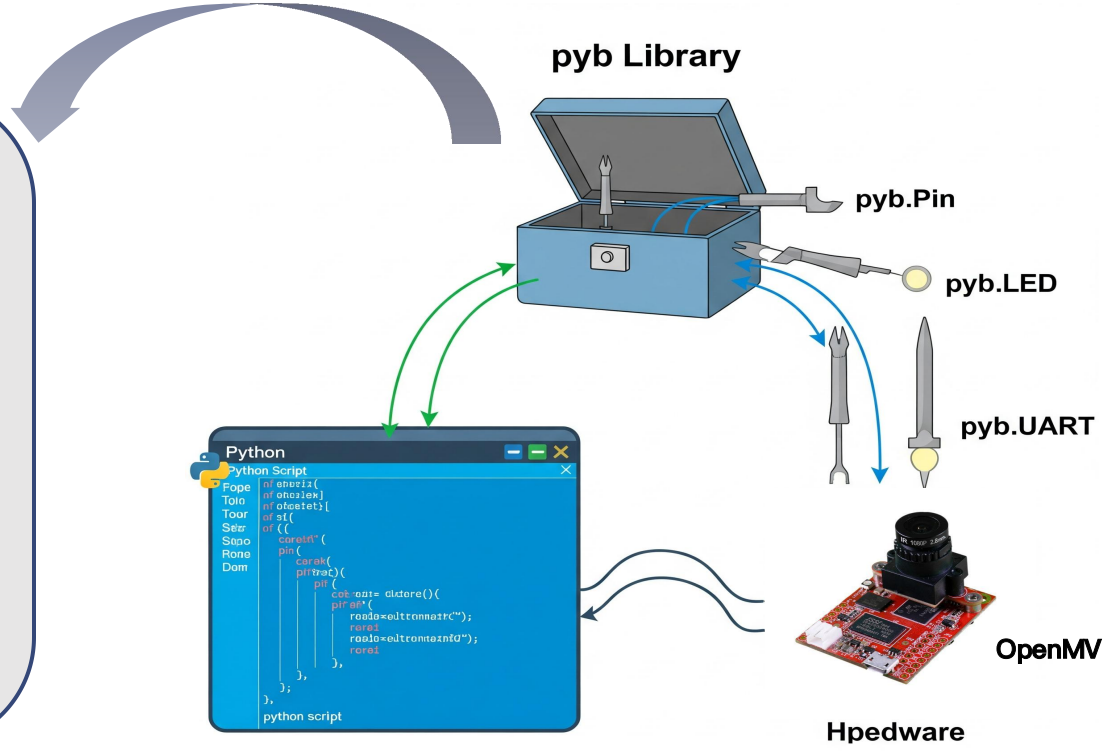
Implementation of OpenMV serial communication

2.1 pyb库

pyb 是 MicroPython 的核心硬件驱动库，提供了直接访问和控制微控制器（MCU）底层硬件外设的能力，可以把它想象成一个专门用来操作 OpenMV 开发板上各种硬件的“遥控器”或“工具箱”，它就像是一座桥梁，它允许你的软件代码去直接、精确地控制这些硬件，让它们按照你的意愿工作。

这个“工具箱”里包含了操作各种硬件的工具（类和函数）：

- pyb.Pin: 控制单个GPIO引脚的输入、输出。
- pyb.LED: 控制板载的LED灯。
- pyb.Timer: 使用硬件定时器，可以用来做PWM输出、精确定时等。
- pyb.ADC: 读取模拟信号（比如光敏电阻的阻值）。
- pyb.I2C, pyb.SPI: 控制I2C和SPI通信总线。
- pyb.UART: 用于控制串口通信



2.2 pyb.UART类

InnoLegend

pyb.UART 是 pyb 库中用于管理和使用通用异步收发器（UART） 的类。串口通信的所有操作都可以通过创建和使用这个类的对象来完成。

创建对象

uart_obj = pyb.UART (id, baudrate, ...)

- **id :** 串口的硬件编号。id = 3是最常用的通用串口，对应引脚为 P4 (TX) 和 P5 (RX)。
- **baudrate :** 波特率
- **timeout (整数):** 读取超时时间，单位为毫秒（ms）。

引脚图查看:

https://docs.singtown.com/micropython/zh/latest/openmvcam/openmvcam/tutorial/io_tutorial.html

发送数据

uart.write(buf)

buf: 要发送的数据，建议写成缓冲区类型的数据，也就是 bytes 或 bytearray。部分较老的micropython版本可能不支持发送一个数字或者一个普通的Python字符串 str。

```
uart.write(b'start')
uart.write(bytearray([0xFE, 255, 0xFD]))
```

b'...' 是创建 bytes 对象的快捷方式

接收数据

data=uart.read([nbytes])

读取数据后返回一个 bytes 对象。如果在 timeout 时间内没有读到数据，则返回 None 。如果提供了 nbytes 参数，会尝试读取指定数量的字节，不提供则读取所有缓冲区数据。

uart.any()

用于检查串口的接收缓冲区里是否有数据在等待被读取，会返回等待被读取的字节数。在 while True 循环中，先用 uart.any() 判断，再用 uart.read() 读取。这可以避免你的主程序因为等待数据而卡住（阻塞），确保程序的流畅性。

2.3 OpenMV串口通信实战

InnoLegend

```
import pyb
import time
# 初始化串口3 (引脚 P4=TX, P5=RX), 波特率 115200
uart=pyb.UART(3, 115200)
defparse_uart_data():
    ifuart.any()>=3:
        frame=uart.read(3)
        ifframe[0] ==0xFEandframe[2] ==0xFD:
            data=frame[1]
            print(f"成功接收指令包: [FE {hex(data)} FD]")
            returnTrue
last_send_time=time.ticks_ms()
whileTrue:
    parse_uart_data()
    iftime.ticks_diff(time.ticks_ms(), last_send_time) >1000:
        last_send_time=time.ticks_ms()  # 更新发送时间
        packet_to_send=bytearray([0xFE,0x01,0xFD])
        uart.write(packet_to_send)
```

第三部分

K210串口通信的实现

Implementation of K210 serial communication

3.1 fm库

K210 片上外设（比如 串口、GPIO、I2C 等）未绑定对应的引脚，非常灵活，但是每个引脚使用之前必须进行注册。K210上没有 pyb 这个库，pyb 是STM32芯片（比如OpenMV）的专用工具箱，K210是完全不同的RISC-V芯片，所以它有自己的一套工具箱。我们主要会用到 machine 和 fm 这两个库。

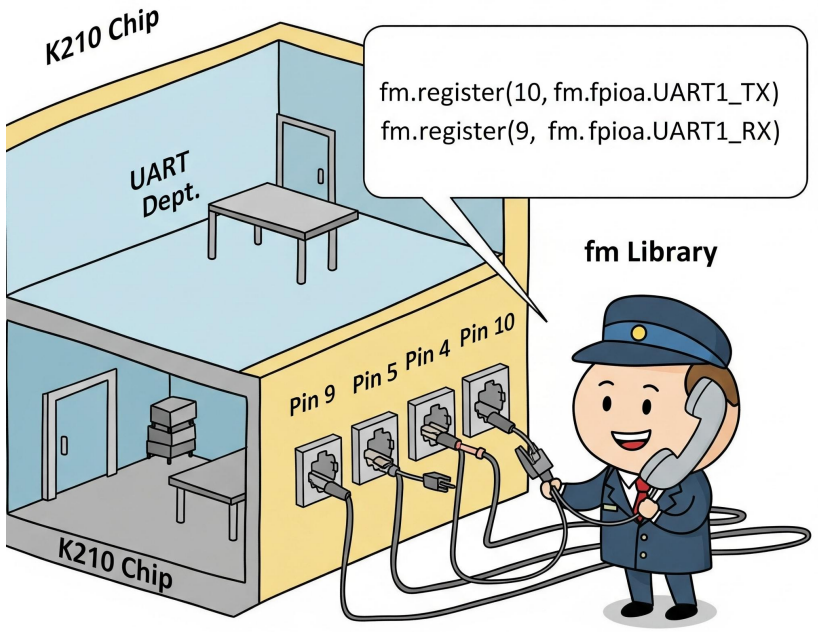
fm 库 ⟷ 电话总机接线员

fm.register(pin, function, force=True)

↓

接线员.连接(外部插口号, 内部部门电话线, force=True)

- pin (外部插口号):物理引脚编号
- function (内部部门电话线):芯片内部功能
- force=True:是否强制分配引脚



3.2 fm库

InnoLegend

常见funtion

功能分类	功能名举例	解释
串口	fm.fpioa.UART1_TX	串口1的发送功能
	fm.fpioa.UART1_RX	串口1的接收功能
GPIO	fm.fpioa.GPIO0	通用输入输出功能0
	fm.fpioa.GPIO1	通用输入输出功能1
I2C	fm.fpioa.I2C1_SDA	I2C总线1的数据线功能
	fm.fpioa.I2C1_SCL	I2C总线1的时钟线功能
SPI	fm.fpioa.SPI0_MISO	SPI总线0的主输入从输出功能

引脚注册示例代码

```
from fpioa_manager import fm
from machine import UART

fm.register(10,
fm.fpioa.UART1_TX,force=True)
fm.register(9,
fm.fpioa.UART1_RX,force=True)

uart = UART(UART.UART1,115200)
uart.write("接线成功，可以通话了！")
```

3.2 machine.UART类

InnoLegend

machine 库是 MicroPython 官方试图提供的一套标准化的硬件访问接口，machine.UART 是这个标准接口中，专门用于控制UART（通用异步收发器/串口）的类。

创建串口对象

```
uart_obj = machine.UART(uart_device, baudrate, ...)
```

- uart_device (串口硬件设备)：如 UART.UART1, UART.UART2，代表芯片内部的第一组还是第二组UART硬件
- baudrate：波特率
- timeout：读取超时，单位毫秒
- read_buf_len：接收缓冲区的大小（字节）

```
#初始化串口
uart1 = UART(UART.UART1,115200,
read_buf_len=4096)

uart2 = UART(UART.UART2,115200,
read_buf_len=4096)
```

3.2 machine.UART类

InnoLegend

发送数据

```
uart.write(buf)
```

- 参数 buf：要发送的数据。建议写成 bytes 或 bytearray格式。

```
uart1.write("Hello\n")# 直接发送字符串
uart1.write(b'Hello Bytes\n')# 发送字节串
packet = bytearray([0xFE,0x01,0xFD])
uart1.write(packet)# 发送字节数组
```

接收数据

```
data = uart.read([nbytes])
```

- 如果不带参数 uart.read()，则读取所有可读数据。如果带参数 nbytes，则尝试读取nbytes个字节。
- 返回值：一个 bytes 对象。如果缓冲区中没有数据可读，它会返回 None

```
data_received = uart.read()
if data_received:
    print("收到了数据:",data_received)
else:
    pass
```

3.3 K210串口通信实战

InnoLegend

```
from fpioa_manager import fm
from machineimportUART
import time
fm.register(9,fm.fpioa.UART1_RX,force=True)
fm.register(10,fm.fpioa.UART1_TX,force=True)
uart=UART(UART.UART1, 115200)
def parse_uart_data():
    if uart.any()>=3:
        frame=uart.read(3)
        if frame[0] ==0xFEandframe[2] ==0xFD:
            data=frame[1]
            print("成功接收指令包: [FE "+hex(data) +" FD]")
            returnTrue
last_send_time=time.ticks_ms()
while True:
    parse_uart_data()
    if time.ticks_diff(time.ticks_ms(), last_send_time) >1000:
        last_send_time=time.ticks_ms()  # 更新发送时间
        packet_to_send=bytearray([0xFE,0x01,0xFD])
        uart.write(packet_to_send)
```

STM32H750 电赛智能车 – 原理与实现

欢迎各位同学交流、指正

InnoLegend 技术团队